Міністерство освіти і науки України

Львівський національний університет імені Івана Франка

Факультет прикладної математики та інформатики

Кафедра програмування

Лабораторна робота №2

РОЗПАРАЛЕЛЕННЯ МНОЖЕННЯ МАТРИЦЬ

з курсу “Паралельні та розподілені обчислення”

Виконав:

студент групи ПМІ-33

Яценко Владислав Тарасович

Львів – 2025

1. Мета роботи

Мета цієї роботи – дослідити методи множення матриць, реалізувати їх **послідовно** та **паралельно**, заміряти час виконання та проаналізувати ефективність паралельних обчислень.

---

2. Теоретична частина

2.1 Послідовне множення

Добуток двох матриць C=A×B

$$C[i][j] = \sum_{k=1}^{m} A[i][k] \cdot B[k][j]$$

Обчислювальна складність для квадратних матриць розмірності n×n:

$$O(n^3)$$

2.2 Паралельне множення

Ідея – розподілити обчислення рядків матриці результату між кількома потоками.

- При k потоках кожен отримує приблизно n/k рядків.
- Якщо n не кратне k, перші потоки обробляють на один рядок більше.
- Результати синхронізуються в головному потоці.

2.3 Закон Амдала

Навіть якщо розпаралелити обчислення, існують частини коду, які **залишаються послідовними** (ініціалізація, створення потоків, синхронізація).
Нехай:

- P — частка коду, що виконується паралельно;
- (1−P) — послідовна частина;
- k — кількість потоків.

Тоді максимальне прискорення за законом Амдала:

$$S_{max}(k) = \frac{1}{(1-P) + \frac{P}{k}}$$

Висновок: навіть при нескінченній кількості потоків максимальне прискорення обмежене величиною:

$$S_{max} = \frac{1}{1 - P}$$

Тобто, якщо лише 90% коду можна паралелити, то максимальне прискорення = 10×, навіть при тисячах потоків.

2.4 Метрики оцінки

- **Прискорення:**

$$S = \frac{T_{seq}}{T_{par}}$$

- **Ефективність:**

$$E = \frac{S}{k} \cdot 100\%$$

3. Алгоритм виконання

1. Генеруємо матриці розмірності n×n.
2. Виконуємо послідовне множення, заміряємо час.
3. Виконуємо паралельне множення:
    - ділимо рядки між потоками;
    - запускаємо потоки;
    - синхронізуємо результат.
4. Обчислюємо S, E, аналізуємо дані з точки зору закону Амдала.

5. Результати

1) Для матриць 1000x1000 та 12 потоків

```
Введіть кількість стовпців матриці A / рядків матриці B: 1000
Введіть кількість стовпців матриці B: 1000
Введіть кількість потоків k: 12

=== Розмір матриць ===
A: 1000 × 1000
B: 1000 × 1000
Потоків: 12

Розподіл рядків: 83 базових + 4 додаткових

Виконання 3 прогонів для найкращого результату...
Прогон 1/3:
  Послідовно: 4170,02 ms, Паралельно: 749,99 ms
Прогон 2/3:
  Послідовно: 4243,43 ms, Паралельно: 732,81 ms
Прогон 3/3:
  Послідовно: 4287,22 ms, Паралельно: 824,37 ms

Найкращі результати:
Час послідовного множення: 4170,02 ms
Час паралельного множення: 732,81 ms
Результати співпадають
```

```
Статистика операцій:
Загальна кількість операцій: 1 000 000 000
Потік 1: 84 000 000 операцій
Потік 2: 84 000 000 операцій
Потік 3: 84 000 000 операцій
Потік 4: 84 000 000 операцій
Потік 5: 83 000 000 операцій
Потік 6: 83 000 000 операцій
Потік 7: 83 000 000 операцій
Потік 8: 83 000 000 операцій
Потік 9: 83 000 000 операцій
Потік 10: 83 000 000 операцій
Потік 11: 83 000 000 операцій
Потік 12: 83 000 000 операцій

Прискорення: 5,69×
Теоретичний максимум: 12,00×
Ефективність: 47,4 %
Реальна паралельна частка P ≈ 89,9 %
Максимум прискорення за Амдалем (∞ потоків): 9,92×
```

2) Для матриць 1000x1000 та 2 потоків

```
Розподіл рядків: 500 базових + 0 додаткових

Виконання 3 прогонів для найкращого результату...
Прогон 1/3:
  Послідовно: 4624,62 ms, Паралельно: 2394,89 ms
Прогон 2/3:
  Послідовно: 4397,57 ms, Паралельно: 2310,14 ms
Прогон 3/3:
  Послідовно: 4840,16 ms, Паралельно: 4360,82 ms

Найкращі результати:
Час послідовного множення: 4397,57 ms
Час паралельного множення: 2310,14 ms
Результати співпадають

Статистика операцій:
Загальна кількість операцій: 1 000 000 000
Потік 1: 500 000 000 операцій
Потік 2: 500 000 000 операцій

Прискорення: 1,90×
Теоретичний максимум: 2,00×
Ефективність: 95,2 %
Реальна паралельна частка P ≈ 94,9 %
Максимум прискорення за Амдалем (∞ потоків): 19,75×
```

Висновок:

У цій роботі я порівняв послідовне та паралельне множення матриць. Паралельний варіант працює значно швидше, але з ростом кількості потоків ефективність поступово падає через накладні витрати та послідовну частину коду. Результати загалом підтвердили закон Амдала і показали, що багатопоточність реально корисна для великих обчислень, хоча має свої межі.